

BCIS Subscription Billing and Collection System

Technical Documentation

Subscription billing, collection and subscriber ledger system for Bukidnon Cable and Internet Services.

Version 1.0.0 · Electron desktop clients → Fastify API → PostgreSQL

1. Business Analysis

1.1 The business

Bukidnon Cable and Internet Services (BCIS) sells Internet, Cable and Combo subscriptions. Subscribers are billed every month and pay in one of three ways: at the office, to a field collector who visits house to house, or by GCash (the customer sends a screenshot of the transaction to the company Facebook Page). Three office PCs must use the system at the same time.

1.2 Problems the system solves

Problem in the manual process	How the system addresses it
Balances are recomputed by hand and drift over time	Every charge and payment is an entry in an append-only subscriber ledger; the balance is always recomputed from those entries
Partial and advance payments are hard to track	Payments are allocated to invoices through allocation records; any excess is kept as advance credit and applied automatically to later invoices
A GCash screenshot can be credited twice, or credited without checking	Proofs go into a verification queue; only an authorized verifier can post them; a reference number can back only one posted payment
Collector cash is not reconciled	Collection batches record what was collected, what was remitted and the exact shortage or overage, with a mandatory explanation
Mistakes are "fixed" by editing or deleting records	Posted payments are reversed, invoices are voided or adjusted; nothing financial is deleted and every change is audited
The owner cannot see receivables	Dashboard, AR aging, overdue list, suspension candidates and 16 exportable reports
Three PCs sharing one file corrupt data	One central API server and PostgreSQL; the PCs are clients only

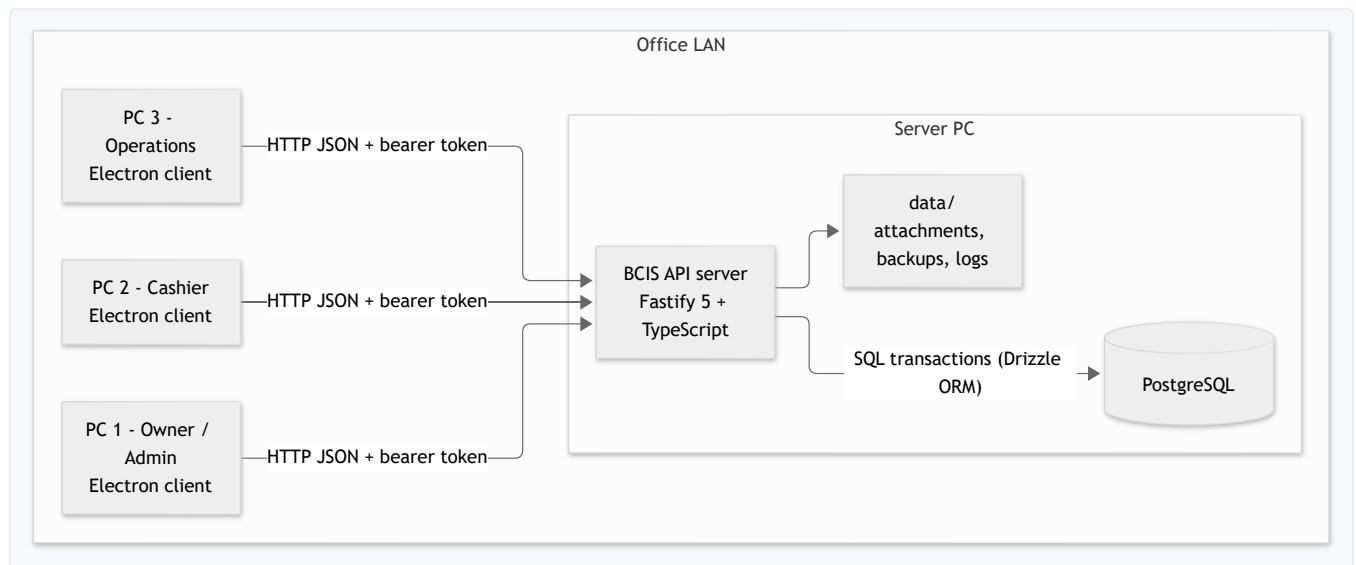
1.3 Users and roles

Role	Purpose	Sensitive permissions it does not have
Owner / Super Admin	Everything, including users, settings, backup and restore	—
Administrator	Subscribers, plans, services, billing, collections, reversals, closing batches	users, settings, backup, restore
Cashier	Find subscriber, receive payment, print receipt, record GCash proofs	reverse payment, verify GCash, manual allocation, billing
Collection Supervisor	Areas, collectors, batches, remittance, reconciliation	close batch, reverse payment, billing
Accounting / Auditor	Read-only finance, reports, audit trail	every mutation
Technician	Service accounts, suspensions, complete reconnection work	all financial data
Read-only Viewer	Dashboards and reports	every mutation, audit trail

The complete matrix (34 permissions × 7 roles) is defined in `source/shared/src/permissions.ts`, loaded into the database by the base seed, shown in **Administration > Roles & Permissions**, and asserted by the acceptance tests.

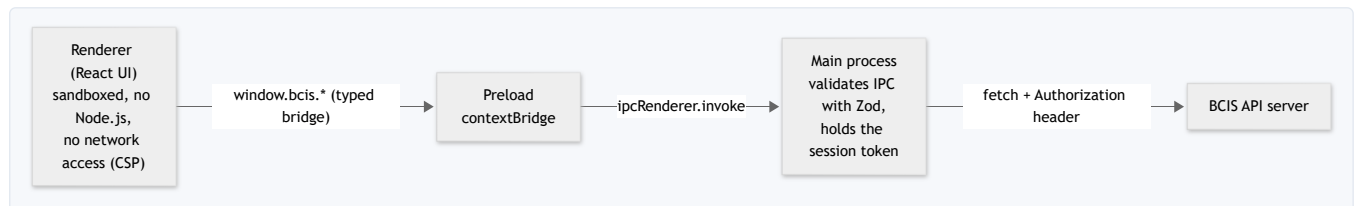
2. Architecture

2.1 Overview



The three office PCs never open the database. All reads and writes go through the API, so validation, authorization, transactions and audit logging are applied in exactly one place.

2.2 Inside one desktop client



- `contextIsolation: true`, `nodeIntegration: false`, `sandbox: true`, `webSecurity: true`.
- The renderer can only call the functions exposed as `window.bcis` (see section 7.2). `ipcRenderer` itself is not exposed.
- The session token is kept in main-process memory only. The renderer never sees it, so script running in the page could not steal it.
- A Content-Security-Policy in production builds forbids remote scripts and any network connection from the renderer (`connect-src 'none'`).
- Navigation to other origins, new windows, `<webview>` and all permission requests are denied.

2.3 Repository layout

Path	Contents
<code>source/shared</code>	Pure domain rules (money, allocation, invoice status, aging, reconciliation), permission matrix, Zod request schemas, response types. Used by both API and UI.
<code>source/api</code>	Fastify server: <code>routes.ts</code> (authorize + validate + delegate), <code>modules/*/service.ts</code> (business rules and SQL), <code>db/schema.ts</code> (Drizzle), <code>lib</code> (audit, sequences, passwords, attachments).
<code>source/desktop</code>	Electron <code>main</code> , <code>preload</code> , and the React renderer (<code>components</code> , <code>pages</code> , <code>lib</code>).
<code>database/migrations</code>	SQL migrations generated by Drizzle Kit, plus a hand-written migration for guard triggers and search indexes.
<code>database/seeds</code>	<code>base.ts</code> (roles, permissions, sequences), <code>demo.ts</code> (synthetic demo dataset), <code>run.ts</code> (CLI).
<code>tests/acceptance</code>	API-level acceptance tests AT-01 to AT-12 and supporting workflows.
<code>tests/e2e</code>	Playwright tests that drive the real Electron app and capture screenshots.
<code>docs</code> , <code>reports-samples</code> , <code>release</code>	Documentation, sample report output, installer output.

2.4 Module boundaries

Business rules live in services and in `@bcis/shared`, never in React components or route handlers:

- **Route handler** — three steps only: `app.can(permission)`, `Schema.parse(input)`, call the service.
- **Service** — opens the transaction, takes locks, applies the rule, writes the audit record.
- **Shared domain function** — pure calculation with no I/O (for example `allocateOldestFirst`), unit tested and reused by the UI for previews.
- **React page** — displays data and collects input. The allocation preview on the Receive Payment screen is computed by the server (`POST /payments/preview`) using the same function the posting uses.

3. Technology Decisions

Layer	Choice	Reason
Desktop shell	Electron 44, electron-vite 5, electron-builder 26	Windows desktop app and NSIS installer from one TypeScript codebase
UI	React 19, TypeScript strict, Tailwind CSS 4, shadcn-style components on Radix primitives	Typed UI; one consistent design system built from the brief's colour tokens
Tables / server state	TanStack Table 8, TanStack Query 5	Server-side paging, sorting and search; cached, refetched queries
Forms / validation	React Hook Form 7 + Zod 4	The same Zod schema validates the form in the client and the request on the server
API	Fastify 5	Small, fast, typed; one process serves all three PCs
Database	PostgreSQL 17	Real concurrent transactions, row locks, constraints, partial unique indexes, triggers
ORM / migrations	Drizzle ORM 0.45 + Drizzle Kit	Typed schema as code; SQL migrations kept in the repository
Reports	pdfmake 0.3 (PDF), ExcelJS 4 (XLSX)	Server-side rendering so every PC produces identical output
Tests	Vitest 5, Playwright 1.63 (Electron)	Unit, API integration and desktop end-to-end
Logging	Pino 10	Structured JSON logs with secret redaction

Why PostgreSQL instead of a shared SQLite file. A SQLite file on a network share relies on file locking over SMB, which is unreliable and is a known cause of database corruption; it allows only one writer at a time; and every PC would need direct write access to the whole database file, so no server-side authorization would be possible. PostgreSQL is a server: it gives row-level locking, true concurrent transactions, constraints enforced centrally, and the clients only ever talk to the API.

Money. Every amount is an integer number of centavos stored as `BIGINT` (`₹999.00` is `99900`). No floating-point arithmetic is used for money anywhere: user input is parsed by string arithmetic (`parseMoney`), and an ESLint rule bans `parseFloat`. Spreadsheet and PDF output divide by 100 only for display.

Password hashing. `scrypt` (Node.js built-in) with $N=32768$, $r=8$, $p=3$ and a random 16-byte salt per password; the parameters are stored with the hash. No native add-on is needed, which keeps deployment simple.

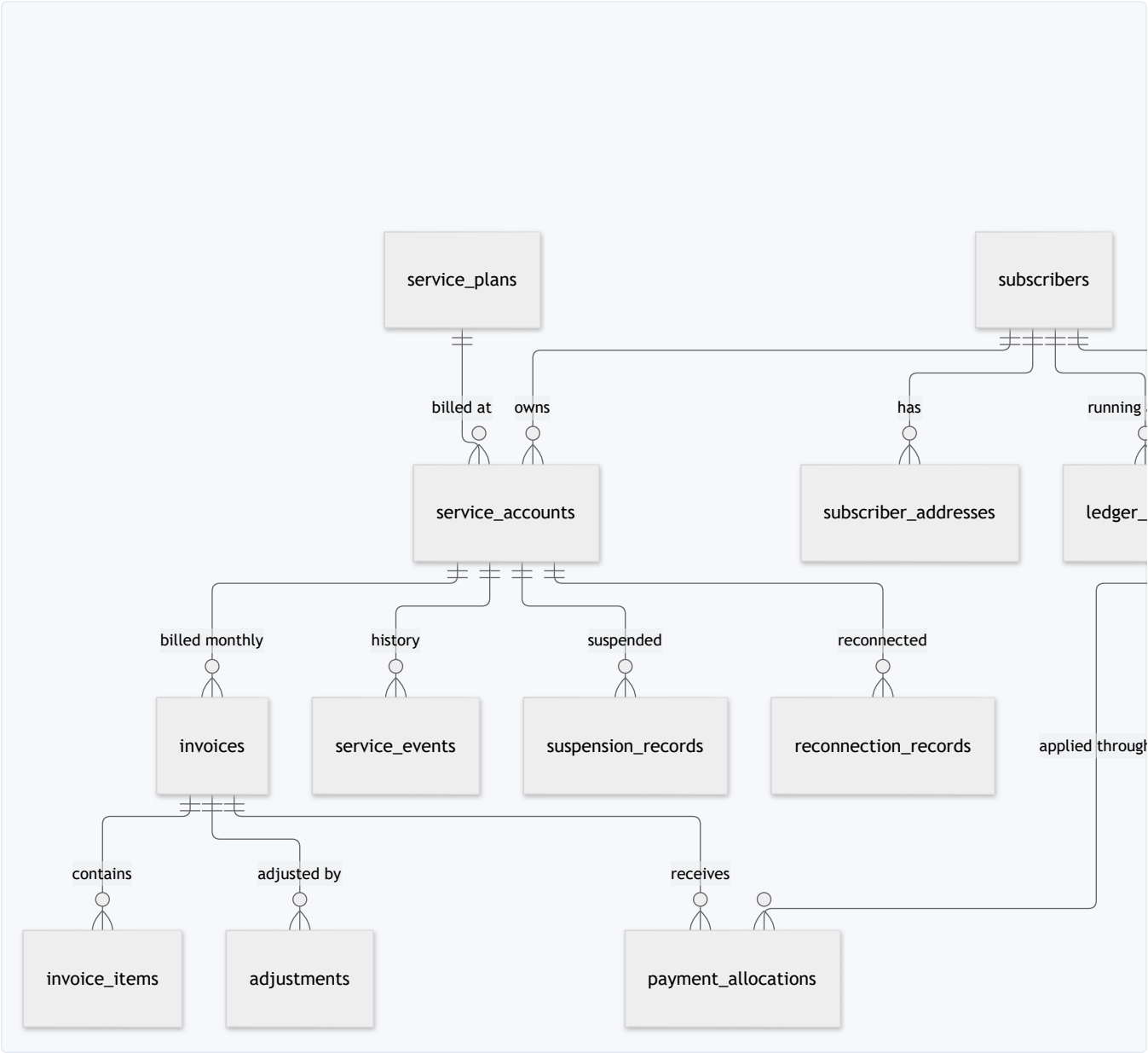
4. Modules

Module	Service file	Responsibilities
Authentication	<code>modules/auth/service.ts</code>	Login, lockout after failed attempts, opaque bearer sessions (only the SHA-256 of the token is stored), idle lock, unlock, logout, change password
Users and roles	<code>modules/users/service.ts</code>	User CRUD, role assignment, password reset (revokes sessions)
Master data	<code>modules/masterdata/service.ts</code>	Plans (price changes apply to future billing), collection areas, collectors and their area assignments
Subscribers	<code>modules/subscribers/service.ts</code>	Registration, addresses, search, status changes (never deleted)
Services	<code>modules/services/service.ts</code>	Service accounts, plan change, suspension, reconnection workflow, termination, service history
Billing and ledger	<code>modules/billing/*</code>	Monthly generation, draft/finalize, invoice numbering, void, adjustments, advance-credit application, ledger queries
Payments	<code>modules/payments/service.ts</code>	Allocation preview, posting, receipts, reversal
GCash	<code>modules/gcash/service.ts</code>	Proof submission, duplicate detection, verify (posts payment), reject
Collections	<code>modules/collections/service.ts</code>	Route sheets, batches, collection recording, remittance, reconciliation, closing
Receivables	<code>modules/receivables/service.ts</code>	Outstanding, overdue, aging, suspension candidates
Dashboard	<code>modules/dashboard/service.ts</code>	KPIs and chart data
Reports	<code>modules/reports/*</code>	16 report definitions and the PDF/XLSX/CSV renderers
Administration	<code>modules/admin/*</code>	Audit log, integrity check, backup, verify, restore, global search

5. Data Model

The entity-relationship diagram is in [docs/erd.pdf](#) (generated from the foreign keys of the live database). The full data dictionary is Appendix A.

5.1 Core relationships



5.2 Integrity enforced by the database

These rules hold even if application code has a bug or someone uses a SQL prompt.

Rule	Mechanism
Unique subscriber, service account, invoice, receipt, adjustment, reversal, batch and remittance numbers	UNIQUE constraints
One live invoice per service account and billing period	Partial unique index <code>invoices_account_period_uq</code> (<code>WHERE status <> 'VOID'</code>)
One posted payment per GCash reference	Partial unique index on <code>upper(reference_no)</code> <code>WHERE method = 'GCASH' AND status = 'POSTED'</code>
A payment is reversed at most once; a retried request posts once	UNIQUE on <code>payment_reversals.payment_id</code> and <code>payments.idempotency_key</code>
<code>balance = total + adjustments - paid</code> , never negative	CHECK constraints on <code>invoices</code>
A ledger entry is exactly one side	CHECK <code>((debit = 0) <> (credit = 0))</code>
Unapplied credit never exceeds the payment	CHECK on <code>payments</code>
A reconciled or closed batch carries <code>difference = remitted - collected</code> , the matching variance type, and a reason when not balanced	CHECK <code>collection_batches_variance_ck</code>

Rule	Mechanism
Payments, allocations, receipts, reversals, ledger entries, adjustments, remittances and audit rows cannot be deleted	BEFORE DELETE triggers (bcis_forbid_delete)
Ledger entries, audit rows, reversals and adjustments cannot be updated	BEFORE UPDATE triggers (bcis_forbid_update)
A posted payment's amount, date, method, reference and subscriber cannot be edited; REVERSED is final	Trigger bcis_guard_payment
Receipt numbers cannot be reassigned; a void receipt cannot be re-issued	Trigger bcis_guard_receipt
A finalized invoice's lines and totals cannot change; it cannot be deleted	Triggers bcis_guard_invoice, bcis_guard_invoice_item
Every owned record references its owner	Foreign keys on all child tables

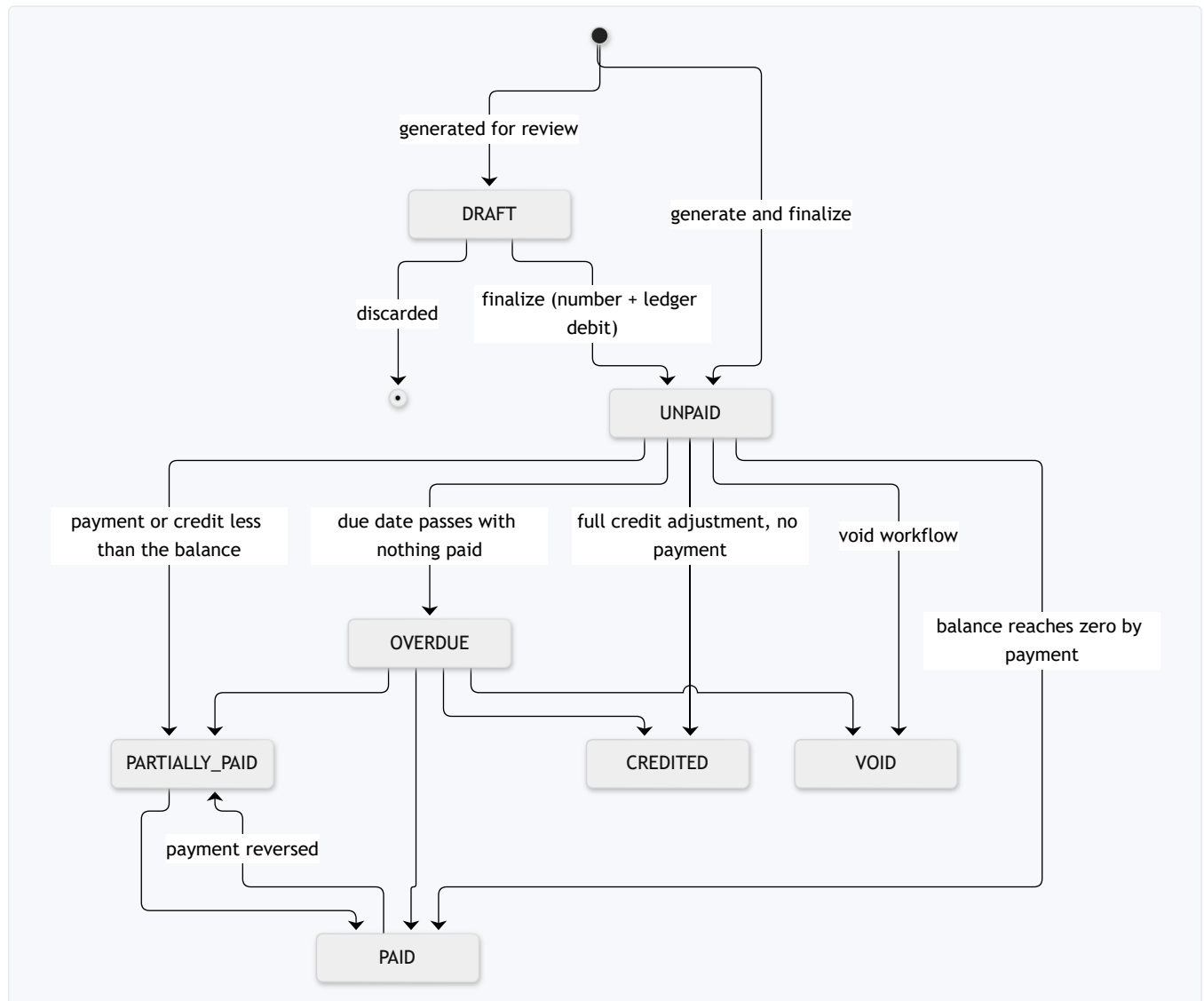
5.3 Indexes for the target volumes

The target is 20,000 subscribers, 500,000 invoices, 500,000 payments and 1,000,000 ledger entries. Lists are always paged on the server (25–200 rows) and never loaded whole into the client.

- Subscriber search: trigram GIN indexes on name, contact number and address (`pg_trgm`) make "contains" searches fast; b-tree on account number, area, collector and status.
- Invoices: (`due_date`, `status`), (`subscriber_id`), (`billing_period`) and a partial index on open invoices (`subscriber_id`, `due_date`) WHERE `balance > 0`.
- Payments: (`subscriber_id`, `payment_date`), (`payment_date`), (`reference_no`), (`collector_id`), (`collection_batch_id`).
- Ledger: (`subscriber_id`, `entry_date`, `id`) — exactly the order the running balance is computed in.
- Audit: (`at`), (`entity_type`, `entity_id`), (`actor_id`, `at`), (`subscriber_id`).

6. Financial Rules

6.1 Invoice life cycle



The status is derived by one function, `deriveInvoiceStatus` in `@bcis/shared`, from the amounts and the due date. Receivable aging never relies on the status: it is always computed from due date and balance, so a partially paid invoice that is past due is still aged correctly.

6.2 Monthly billing

- One invoice per **active** service account whose billing start date is on or before the end of the period. Suspended and terminated accounts are not billed.
- Line items: subscription at the account's **current rate** (copied onto the invoice), installation fee on the first invoice, optional late-payment penalty, optional monthly discount.
- A plan price change updates the rate of its service accounts for **future** billing; invoices already generated keep their amounts because the price is stored on the invoice line.
- **No duplicates:** the insert uses `ON CONFLICT DO NOTHING` against the partial unique index. Running the generation twice, or from two PCs at once, creates each invoice exactly once (AT-11, AT-09).
- Each invoice (header, lines, number, ledger debit, credit application) commits in its own transaction, so a run that is interrupted leaves only complete invoices and can simply be run again.
- Limitation: no proration — a service is billed for the whole month in which its billing starts.

6.3 Payment posting and allocation

A payment is posted in **one database transaction**:

1. lock the subscriber row (`SELECT ... FOR UPDATE`) — serializes all financial activity of that subscriber;
2. lock the subscriber's open invoices, oldest first;
3. compute the allocation (`allocateOldestFirst`, or `allocateManually` for an authorized override);
4. insert the payment and its allocation rows; update each invoice's paid amount, balance and status;
5. take the next receipt number and insert the receipt;

6. post one ledger credit for the full amount;
7. write the audit record.

If anything fails, or the server stops halfway, PostgreSQL rolls the whole transaction back: there is never a payment without its receipt and ledger entry, and the receipt number is not consumed. If the client retries, the idempotency key guarantees the payment is posted once.

Default allocation rule — oldest unpaid invoice first: earliest due date, then earliest invoice date, then lowest id.

Partial payment is simply an allocation smaller than the invoice balance: one `payment_allocations` row for ₱500.00 against a ₱999.00 invoice leaves `amount_paid = 50000`, `balance = 49900`, `status PARTIALLY_PAID`.

Advance payment policy: money beyond the open invoices is not lost and not forced onto anything. It stays on the payment as `unapplied_amount` (the subscriber's advance credit) and the ledger balance goes negative by that amount. When later invoices are finalized, the credit is applied automatically — oldest payment first, oldest invoice first — as allocation rows of type `CREDIT`. Invariant, checked by the integrity check: for every posted payment, $allocations + unapplied = amount$.

Manual allocation requires the `payment.allocate_manual` permission; each line must target an open invoice of the same subscriber and may not exceed its balance.

6.4 Reversal instead of deletion

`POST /payments/:id/reverse` (permission `payment.reverse`, reason required) in one transaction:

- marks the payment `REVERSED` (the row stays);
- marks its allocations reversed and restores each invoice's paid amount, balance and status;
- voids the receipt — the number stays reserved forever and is never issued again;
- inserts a `payment_reversals` row (who, when, why) and a ledger **debit** that offsets the original credit;
- writes an audit record with old and new values.

Deleting would destroy the evidence of what happened and leave gaps in receipt numbering; reversing keeps the full history auditable, which is what an auditor and the tax authority expect.

6.5 Subscriber ledger

`ledger_entries` is append-only: an invoice posts a debit, a payment posts a credit, a reversal posts a debit, an adjustment posts either, a void posts a credit. The running balance is **never stored**; it is recomputed every time with a window function ordered by `(entry_date, id)`, so the same entries always give the same balances.

Proof that the ledger is right — the integrity check (`GET /integrity-check`, also run after every restore and by the demo seed) verifies ten independent rules from the raw tables, including: for every subscriber, $ledger\ balance = open\ invoice\ balances - unapplied\ credit$; every finalized invoice and every payment appears in the ledger exactly once for the right amount; invoice paid amounts equal their non-reversed allocations.

6.6 Document numbering

`number_sequences` holds one row per document type. `nextNumber()` runs `UPDATE ... SET next_value = next_value + 1 ... RETURNING` inside the posting transaction. The row lock makes concurrent issuers queue, so two PCs can never receive the same number; because it is part of the transaction, a rollback returns the number (no gaps). Formats: `BCIS-00001`, `SA-000001`, `INV-000001`, `RCPT-000001`, `ADJ-00001`, `REV-00001`, `CB-00001`, `REM-00001`.

6.7 GCash verification policy

1. Staff records the proof: reference number, sender, amount, date and the image. **Nothing is posted.**
2. The system lists every other proof or payment with the same reference number (warning).
3. A user with `gcash.verify` compares the proof with the real GCash transaction history and verifies or rejects it; the verifier and time are stored.
4. Verification posts and allocates the payment in the same transaction as the status change.
5. A reference that already backs a **posted** payment is **blocked** — in the service and by the unique index. Reversing that payment frees the reference.
6. Users without `gcash.verify` cannot post a GCash payment directly from Receive Payment.

6.8 Collection batches and reconciliation

Life cycle: `OPEN` → `IN_PROGRESS` → `SUBMITTED` → `REMITTED` → `RECONCILED` → `CLOSED`.

- Creating a batch snapshots every assigned subscriber in the area with an amount due (current bill, arrears, total).
- Each collection is a normal posted payment tagged with the batch and collector.
- Only **cash** must be remitted; GCash and other non-cash collections are reported separately.
- $difference = cash\ remitted - cash\ collected$. Zero is `BALANCED`, negative is `SHORTAGE`, positive is `OVERAGE`.
- A batch that is not balanced **cannot be reconciled without a recorded reason**, and a batch cannot be closed unless it is reconciled. Closing needs a different permission (`collection.close`) and an explicit confirmation. The database `CHECK` refuses a "balanced" result for unequal amounts.

6.9 Suspension and reconnection

- Settings: grace period (default 15 days) and suspension threshold (default 2 unpaid invoices beyond the grace period).
- Suspending records reason, effective date, arrears at suspension and the approver.

- With "reconnection requires settled arrears" on, a reconnection request is refused until the overdue invoices are paid. The plan's reconnection fee is billed as a one-time invoice (so it flows through the ledger), a technician can be assigned, and completion sets the account back to Active.
- Every state change is written to `service_events`.

7. Interfaces

7.1 HTTP API

All routes are under `/api`. Requests and responses are JSON. Every route except `/health` and `/auth/login` needs `Authorization: Bearer <token>`. Errors have one shape: `{ "error": { "code", "message", "fields"? } }`.

Area	Endpoints	Permission
Health	GET <code>/health</code>	none
Auth	POST <code>/auth/login</code> , GET <code>/auth/me</code> , POST <code>/auth/logout</code> , <code>/auth/lock</code> , <code>/auth/unlock</code> , <code>/auth/change-password</code>	session
Lookups, search	GET <code>/lookups</code> , GET <code>/search?q=</code>	session (results filtered by permission)
Dashboard	GET <code>/dashboard</code>	dashboard.view
Users	GET/POST <code>/users</code> , PATCH <code>/users/:id</code> , POST <code>/users/:id/reset-password</code> , GET <code>/roles</code>	user.manage
Plans	GET <code>/plans</code> ; POST <code>/plans</code> , PATCH <code>/plans/:id</code>	plan.view / plan.manage
Subscribers	GET/POST <code>/subscribers</code> , GET/PATCH <code>/subscribers/:id</code> , POST <code>.../addresses</code> , GET <code>.../ledger</code> , <code>.../service-events</code> , <code>.../collections</code>	subscriber.*, billing.view, service.view, collection.view
Service accounts	GET/POST <code>/service-accounts</code> , PATCH <code>.../:id</code> , POST <code>.../:id/suspend</code> , <code>.../reconnection</code> , <code>.../terminate</code> , GET <code>/suspensions</code> , GET <code>/reconnections</code> , POST <code>/reconnections/:id/complete</code>	service.*
Billing	GET <code>/billing/current</code> , <code>/billing/cycles</code> , <code>/billing/preview</code> ; POST <code>/billing/generate</code> , <code>/billing/finalize-drafts</code> , <code>/billing/discard-drafts</code>	billing.view / billing.generate
Invoices	GET <code>/invoices</code> , GET <code>/invoices/:id</code> , POST <code>/invoices/:id/void</code> , GET/POST <code>/adjustments</code>	billing.view, invoice.void, adjustment.create
Payments	GET <code>/payments</code> , GET <code>/payments/:id</code> , POST <code>/payments/preview</code> , POST <code>/payments</code> , POST <code>/payments/:id/reverse</code> , POST <code>/payments/:id/printed</code>	payment.view, payment.create, payment.reverse
GCash	GET/POST <code>/gcash/proofs</code> , GET <code>.../:id</code> , GET <code>.../:id/file</code> , GET <code>/gcash/check-reference</code> , POST <code>.../:id/verify</code> , POST <code>.../:id/reject</code>	gcash.submit, gcash.verify
Collections	GET/POST/PATCH <code>/areas</code> , <code>/collectors</code> ; GET <code>/collections/route-sheet</code> ; GET/POST <code>/batches</code> , POST <code>/batches/:id/start</code> , <code>/collections</code> , <code>/outcomes</code> , <code>/submit</code> , <code>/remittances</code> , <code>/reconcile</code> , <code>/close</code>	collection.*
Receivables	GET <code>/receivables/summary</code> , <code>/outstanding</code> , <code>/overdue</code> , <code>/aging</code> , <code>/suspension-candidates</code>	receivable.view
Reports	GET <code>/reports</code> , GET <code>/reports/:key?format=...</code> (json, pdf, xlsx or csv)	report.view, report.export; audit reports need audit.view
Administration	GET/PUT <code>/settings</code> , GET <code>/audit-logs</code> , GET <code>/integrity-check</code> , GET/POST <code>/backups</code> , POST <code>/backups/:id/verify</code> , POST <code>/backups/:id/restore</code>	settings.manage, audit.view, backup.create, backup.restore

7.2 IPC bridge (window.bcis)

Defined in `source/desktop/src/shared/bridge.ts`; every argument is validated with Zod in the main process and the calling frame's origin is checked.

Function	Purpose
<code>api.request({ method, path, query, body })</code>	Proxies one API call. <code>path</code> must match a strict pattern; the token is added in the main process.
<code>api.download({ path, query })</code>	Fetches a report file and saves it through the native Save dialog.
<code>api.blob({ path })</code>	Fetches a payment-proof image for preview (only <code>/gcash/proofs/:id/file</code>).
<code>auth.login</code> , <code>auth.logout</code> , <code>auth.current</code>	Session handling. <code>login</code> returns the user, never the token.
<code>config.get</code> , <code>config.test</code> , <code>config.setServerUrl</code>	Server address of this client.
<code>print.print</code> , <code>print.savePdf</code>	Print the current print sheet or save it as PDF.
<code>files.saveText</code>	Save a CSV export (<code>.csv</code> names only).
<code>app.info</code>	Version information.

8. Security Model

Topic	Implementation
Authentication	Username + password; bcrypt hashes; generic "incorrect username or password"; constant-time behaviour for unknown users
Failed logins	Configurable lockout (default 5 attempts → 15 minutes); every failure is audited
Sessions	256-bit random opaque token; only its SHA-256 is stored; expiry (12 h), server-side idle lock (30 min), client idle lock (10 min, configurable), manual lock, revocation on logout, password reset and deactivation
Authorization	<code>app.can(...)</code> pre-handler on every protected route reads the user's permissions from the database on each request. The UI hides what a user cannot do, but the tests call the API directly to prove the server refuses (AT-10)
Validation	Zod schemas on every body, query and IPC argument; one error handler maps failures to readable messages
Audit	<code>audit_logs</code> : actor, action, record, reason, old and new values, client IP, time. Written in the same transaction as the change. Append-only by trigger; no API or screen can edit it
Uploads	Type decided from the file's leading bytes (PNG, JPEG, WEBP, PDF), 5 MB limit, stored under a server-generated UUID name; the client file name is only a label; path traversal is impossible because stored names must match a strict pattern
Secrets	<code>.env</code> is git-ignored; <code>npm run setup</code> generates random local secrets; logs redact <code>authorization</code> , <code>password</code> , <code>token</code> and file data; request bodies are not logged
Electron	Sandbox, context isolation, no Node integration, CSP, no navigation or new windows, permission requests denied, DevTools disabled in packaged builds
SQL injection	All values are bound parameters; sort columns come from a whitelist
Transport	HTTP on the office LAN. See limitations (section 12)

9. Reliability and Concurrency

- **Transactions.** Payment posting, reversal, invoice finalization, void, adjustment, GCash verification, batch transitions, reconnection and plan price changes each run in one transaction.
- **Lock order** (prevents deadlocks): subscriber row → invoice rows → number sequence. The subscriber lock is taken before any invoice is inserted, so concurrent sessions queue instead of deadlocking.
- **Three-PC test.** AT-09 starts the API on a real TCP port and has three logged-in clients post 16 payments (five of them to the same invoice), generate the same billing period from two clients and read reports, all at once. It then checks: unique gapless receipt numbers, the exact invoice balance, one invoice per account, a clean integrity check, and independent sessions.
- **Errors.** Expected failures return a clear message and code; unexpected ones return a generic message and are logged with the request id.
- **Overdue status** is refreshed at server start and every 10 minutes.

10. Backup and Restore

A backup is a folder under `data/backups/<name>/` on the server PC:

File	Content
<code>database.dump</code>	<code>pg_dump</code> custom-format archive (sessions and the backup catalogue are excluded)
<code>attachments/</code>	Copy of the payment-proof files
<code>manifest.json</code>	Name, creator, SHA-256 and size of the dump, attachment count, row counts of the main tables

- **Verify** recomputes the checksum, compares it with the manifest and the catalogue, and asks `pg_restore --list` to read the archive.
- **Restore** (Owner only; type `RESTORE` and give a reason): verify again → automatic *pre-restore* safety backup of the current state → `pg_restore --clean --single-transaction --exit-on-error` (all or nothing; on failure the live data is unchanged) → restore attachments → **integrity check** → record the result in the backup history and audit trail. The owner's session survives; other users sign in again.
- Copy `data/backups` to an external drive or another PC regularly: a backup on the same disk does not protect against disk failure. A scheduled-task example is in the deployment guide.

11. Testing Strategy

Level	Tool	What it covers
Unit	Vitest (<code>source/shared/src/domain/domain.test.ts</code>)	Money parsing, allocation (including a 200-case conservation check), invoice state machine, aging boundaries, remittance variance, role matrix
API acceptance	Vitest + Fastify inject + real PostgreSQL (<code>tests/acceptance</code>)	AT-01 to AT-12 and supporting workflows through HTTP, authentication, authorization, validation, services, constraints and triggers
Desktop end-to-end	Playwright driving Electron (<code>tests/e2e</code>)	The live-demo sequence: login, dashboard, profile, ledger/SOA, billing, aging, GCash duplicate, shortage reconciliation, reports export, cashier payment and receipt, role restriction, reversal, lock, backup
Data integrity	<code>GET /integrity-check</code>	Ten cross-checks on real data; run by the seed, after restore and on demand

Results are in `tests/acceptance-test-report.pdf` ; defects found during development are in `tests/bug-log.md` .

12. Limitations and Future Work

- **Transport security.** Client–server traffic is plain HTTP on the office LAN. Before exposing the API beyond a trusted LAN, put it behind TLS (a reverse proxy or Fastify's `https` option).
- **No proration** for services that start mid-month, and no automatic penalty schedule beyond a fixed optional penalty.
- **One business timezone** (configurable, default Asia/Manila).
- **Document preview.** PDF payment proofs are stored and validated but not rendered in the verification pane.
- **Backups are on the server PC;** off-site copies are a manual or scheduled-task step.
- **Unsigned installer.** Windows SmartScreen will warn until the installer is code-signed.

Collector mobile / PWA — what would change

The current design already keeps all rules in the API, so a mobile client would be another client of the same API. Changes needed:

1. **Collector identity:** link `collectors` to `users` and add a `COLLECTOR` role limited to its own batches (`collection.record` scoped by collector id).
2. **Transport:** TLS and reachability outside the LAN (VPN or a reverse proxy), token refresh, device registration and revocation.
3. **Offline queue:** collections recorded offline carry a client-generated idempotency key (already supported by `POST /batches/:id/collections`) and are synchronized when online; the server stays authoritative and rejects stale or conflicting entries explicitly.
4. **Provisional receipts:** a batch-scoped provisional number offline, replaced by the official receipt number at synchronization.
5. **Conflict rules:** what happens when the subscriber already paid at the office before the sync — the excess becomes advance credit and the batch shows the exception.
6. **Route data:** downloadable route sheet with only the fields the collector needs (data minimization on a device that can be lost).

13. Defense Questions — Short Answers

Question	Answer	Where in the code
Why PostgreSQL instead of shared SQLite?	Network file locking is unsafe, single writer, no server-side authorization; PostgreSQL gives concurrent transactions, row locks and central constraints	Section 3
Where does authoritative billing/payment logic live?	In API services and pure functions in <code>@bcis/shared</code> ; never in React. One place to secure, test and audit	<code>modules/billing</code> , <code>modules/payments</code> , <code>shared/src/domain</code>
How are duplicate invoices, receipts and GCash transactions prevented?	Partial unique index + <code>ON CONFLICT DO NOTHING</code> ; row-locked number sequence + unique constraint; service check + partial unique index on the reference	<code>db/schema.ts</code> , <code>lib/sequences.ts</code> , <code>payments/service.ts</code>
How is a partial payment represented?	A <code>payment_allocations</code> row smaller than the invoice balance; the invoice keeps <code>amount_paid</code> and <code>balance</code> , status <code>PARTIALLY_PAID</code>	<code>billing/core.ts</code>
Why reverse instead of delete?	History, audit, receipt numbering and the ledger stay complete; the database forbids deleting posted payments	<code>payments/service.ts</code> , migration <code>0001</code>
How do you prove the ledger balance is correct?	Append-only entries, recomputed running balance, and the integrity check reconciling ledger = invoices – credit	<code>admin/integrity.ts</code>
What if the API crashes halfway through posting?	The transaction never commits, PostgreSQL rolls it back, the receipt number is not consumed; a retry with the same idempotency key posts once	<code>payments/service.ts</code>
Server-side authorization versus hiding buttons?	Hiding is convenience; <code>app.can()</code> on every route is the control. Tests call the API directly as a cashier and get 403	<code>plugins/auth.ts</code> , AT-10
How are backups validated and restored?	Checksum + <code>pg_restore --list</code> ; restore is verified first, single-transaction, followed by the integrity check	<code>admin/backup.ts</code> , AT-12
What changes for a collector mobile app?	Section 12	—

Appendix A. Data Dictionary

Generated from the live PostgreSQL catalogue by `npm run docs:build`. Money columns are `bigint` integer centavos.

adjustments

Column	Type	Required	Key	Default
id	integer	yes	PK	<code>nextval('adjustments_id_seq'::regclass)</code>
adjustment_no	<code>varchar(20)</code>	yes		
subscriber_id	integer	yes	FK → subscribers	
invoice_id	integer	yes	FK → invoices	
type	<code>enum adjustment_type</code>	yes		
amount	<code>bigint</code>	yes		
reason	<code>text</code>	yes		
created_by	integer		FK → users	
created_at	<code>timestampz</code>	yes		<code>now()</code>

Constraints:

- **CHECK** `adjustments_amount_ck`: `CHECK ((amount > 0))`
- **UNIQUE** `adjustments_adjustmentNo_unique`: `UNIQUE (adjustment_no)`

Indexes:

- `adjustments_invoice_idx`: `btree (invoice_id)`

application_settings

Column	Type	Required	Key	Default
key	<code>varchar(60)</code>	yes	PK	
value	<code>jsonb</code>	yes		
updated_by	integer		FK → users	
updated_at	<code>timestampz</code>	yes		<code>now()</code>

audit_logs

Column	Type	Required	Key	Default
id	<code>bigint</code>	yes	PK	<code>nextval('audit_logs_id_seq'::regclass)</code>
at	<code>timestampz</code>	yes		<code>now()</code>
actor_id	integer		FK → users	
actor_username	<code>varchar(40)</code>			
action	<code>varchar(60)</code>	yes		
entity_type	<code>varchar(40)</code>	yes		
entity_id	<code>varchar(40)</code>			
subscriber_id	integer			
reason	<code>text</code>			
old_values	<code>jsonb</code>			
new_values	<code>jsonb</code>			
ip	<code>varchar(64)</code>			

Indexes:

- `audit_logs_actor_idx`: `btree (actor_id, at)`
- `audit_logs_at_idx`: `btree (at)`
- `audit_logs_entity_idx`: `btree (entity_type, entity_id)`
- `audit_logs_subscriber_idx`: `btree (subscriber_id)`

backup_history

Column	Type	Required	Key	Default
id	integer	yes	PK	<code>nextval('backup_history_id_seq'::regclas</code>
kind	<code>varchar(10)</code>	yes		
name	<code>varchar(80)</code>	yes		
status	<code>varchar(12)</code>	yes		
size_bytes	<code>bigint</code>	yes		<code>0</code>

Column	Type	Required	Key	Default
checksum	char(64)			
verified_at	timestampz			
verification_result	text			
notes	text			
created_by	integer		FK → users	
created_by_name	varchar(120)			
created_at	timestampz	yes		now()

batch_accounts

Column	Type	Required	Key	Default
id	integer	yes	PK	nextval('batch_accounts_id_seq'::regclas
batch_id	integer	yes	FK → collection_batches	
subscriber_id	integer	yes	FK → subscribers	
sequence	integer	yes		0
current_bill	bigint	yes		0
arrears	bigint	yes		0
total_due	bigint	yes		0
collected_amount	bigint	yes		0
outcome	enum batch_outcome	yes		'PENDING'::batch_outcome
notes	text			

Indexes:

- batch_accounts_subscriber_idx: btree (subscriber_id)
- batch_accounts_uq (unique): btree (batch_id, subscriber_id)

billing_cycles

Column	Type	Required	Key	Default
id	integer	yes	PK	nextval('billing_cycles_id_seq'::regclas
period	char(7)	yes		
period_start	date	yes		
period_end	date	yes		
last_generated_at	timestampz			
last_generated_by	integer		FK → users	
created_at	timestampz	yes		now()

Constraints:

- **UNIQUE** billing_cycles_period_unique: UNIQUE (period)

collection_areas

Column	Type	Required	Key	Default
id	integer	yes	PK	nextval('collection_areas_id_seq'::regcl
code	varchar(20)	yes		
name	varchar(100)	yes		
description	text			
route_notes	text			
is_active	boolean	yes		true
created_at	timestampz	yes		now()

Constraints:

- **UNIQUE** collection_areas_code_unique: UNIQUE (code)

collection_batches

Column	Type	Required	Key	Default
id	integer	yes	PK	nextval('collection_batches_id_seq'::reg
batch_no	varchar(20)	yes		
collector_id	integer	yes	FK → collectors	
area_id	integer	yes	FK → collection_areas	

Column	Type	Required	Key	Default
collection_date	date	yes		
status	enum batch_status	yes		'OPEN'::batch_status
expected_total	bigint	yes		0
cash_collected	bigint	yes		0
non_cash_collected	bigint	yes		0
cash_remitted	bigint	yes		0
difference	bigint	yes		0
variance_type	enum variance_type			
variance_reason	text			
notes	text			
created_by	integer		FK → users	
created_at	timestamptz	yes		now()
started_at	timestamptz			
submitted_at	timestamptz			
reconciled_by	integer		FK → users	
reconciled_at	timestamptz			
closed_by	integer		FK → users	
closed_at	timestamptz			

Constraints:

- **CHECK** collection_batches_variance_ck: CHECK (((status <> ALL (ARRAY['RECONCILED'::batch_status, 'CLOSED'::batch_status])) OR ((variance_type IS NOT NULL) AND (difference = (cash_remitted - cash_collected)) AND ((variance_type = 'BALANCED'::variance_type) = (difference = 0)) AND ((difference = 0) OR (variance_reason IS NOT NULL)))))
- **UNIQUE** collection_batches_batchNo_unique: UNIQUE (batch_no)

Indexes:

- collection_batches_area_idx: btree (area_id)
- collection_batches_collector_idx: btree (collector_id, collection_date)
- collection_batches_status_idx: btree (status)

collector_assignments

Column	Type	Required	Key	Default
id	integer	yes	PK	nextval('collector_assignments_id_seq'::
collector_id	integer	yes	FK → collectors	
area_id	integer	yes	FK → collection_areas	
assigned_from	date	yes		
assigned_to	date			
is_active	boolean	yes		true
assigned_by	integer		FK → users	
created_at	timestamptz	yes		now()

Indexes:

- collector_assignments_active_uq (unique): btree (collector_id, area_id) WHERE is_active
- collector_assignments_area_idx: btree (area_id)
- collector_assignments_collector_idx: btree (collector_id)

collector_remittances

Column	Type	Required	Key	Default
id	integer	yes	PK	nextval('collector_remittances_id_seq'::
remittance_no	varchar(20)	yes		
batch_id	integer	yes	FK → collection_batches	
collector_id	integer	yes	FK → collectors	
amount	bigint	yes		
notes	text			
received_by	integer		FK → users	
remitted_at	timestamptz	yes		now()

Constraints:

- **CHECK** collector_remittances_amount_ck: CHECK ((amount >= 0))
- **UNIQUE** collector_remittances_remittanceNo_unique: UNIQUE (remittance_no)

Indexes:

- `collector_remittances_batch_idx: btree (batch_id)`

collectors

Column	Type	Required	Key	Default
id	integer	yes	PK	<code>nextval('collectors_id_seq'::regclass)</code>
code	varchar(20)	yes		
full_name	varchar(120)	yes		
phone	varchar(20)			
is_active	boolean	yes		<code>true</code>
created_at	timestampz	yes		<code>now()</code>

Constraints:

- **UNIQUE** `collectors_code_unique: UNIQUE (code)`

invoice_items

Column	Type	Required	Key	Default
id	integer	yes	PK	<code>nextval('invoice_items_id_seq'::regclass)</code>
invoice_id	integer	yes	FK → invoices	
item_type	enum invoice_item_type	yes		
description	varchar(200)	yes		
quantity	integer	yes		<code>1</code>
unit_amount	bigint	yes		
amount	bigint	yes		

Constraints:

- **CHECK** `invoice_items_amount_ck: CHECK (((amount = (quantity * unit_amount)) AND (quantity > 0)))`

Indexes:

- `invoice_items_invoice_idx: btree (invoice_id)`

invoices

Column	Type	Required	Key	Default
id	integer	yes	PK	<code>nextval('invoices_id_seq'::regclass)</code>
invoice_no	varchar(20)			
type	enum invoice_type	yes		<code>'MONTHLY'::invoice_type</code>
subscriber_id	integer	yes	FK → subscribers	
service_account_id	integer	yes	FK → service_accounts	
billing_cycle_id	integer		FK → billing_cycles	
billing_period	char(7)			
period_start	date			
period_end	date			
invoice_date	date	yes		
due_date	date	yes		
subtotal	bigint	yes		
discount_total	bigint	yes		<code>0</code>
total	bigint	yes		
adjustments_total	bigint	yes		<code>0</code>
amount_paid	bigint	yes		<code>0</code>
balance	bigint	yes		
status	enum invoice_status	yes		<code>'DRAFT'::invoice_status</code>
finalized_at	timestampz			
finalized_by	integer		FK → users	
voided_at	timestampz			
voided_by	integer		FK → users	
void_reason	text			
created_by	integer		FK → users	
created_at	timestampz	yes		<code>now()</code>

Constraints:

- **CHECK** invoices_amounts_ck: CHECK (((total >= 0) AND (amount_paid >= 0) AND (balance >= 0) AND (total = (subtotal - discount_total))))
- **CHECK** invoices_balance_ck: CHECK (((status = ANY (ARRAY['VOID'::invoice_status, 'DRAFT'::invoice_status])) OR (balance = ((total + adjustments_total) - amount_paid))))
- **CHECK** invoices_finalized_ck: CHECK (((status = 'DRAFT'::invoice_status) OR (invoice_no IS NOT NULL)))
- **UNIQUE** invoices_invoiceNo_unique: UNIQUE (invoice_no)

Indexes:

- invoices_account_period_uq (unique): btree (service_account_id, billing_period) WHERE ((billing_period IS NOT NULL) AND (status <> 'VOID'::invoice_status))
- invoices_due_status_idx: btree (due_date, status)
- invoices_open_idx: btree (subscriber_id, due_date) WHERE ((balance > 0) AND (status = ANY (ARRAY['UNPAID'::invoice_status, 'PARTIALLY_PAID'::invoice_status, 'OVERDUE'::invoice_status])))
- invoices_period_idx: btree (billing_period)
- invoices_status_idx: btree (status)
- invoices_subscriber_idx: btree (subscriber_id)

ledger_entries

Column	Type	Required	Key	Default
id	bigint	yes	PK	nextval('ledger_entries_id_seq'::regclas
subscriber_id	integer	yes	FK → subscribers	
service_account_id	integer		FK → service_accounts	
entry_date	date	yes		
source_type	enum ledger_source	yes		
source_id	integer	yes		
reference_no	varchar(30)	yes		
description	varchar(200)	yes		
debit	bigint	yes		0
credit	bigint	yes		0
created_by	integer		FK → users	
created_at	timestamptz	yes		now()

Constraints:

- **CHECK** ledger_one_side_ck: CHECK (((debit >= 0) AND (credit >= 0) AND ((debit = 0) <> (credit = 0))))

Indexes:

- ledger_source_idx: btree (source_type, source_id)
- ledger_subscriber_date_idx: btree (subscriber_id, entry_date, id)

number_sequences

Column	Type	Required	Key	Default
name	varchar(30)	yes	PK	
prefix	varchar(10)	yes		
padding	integer	yes		6
next_value	bigint	yes		1

payment_allocations

Column	Type	Required	Key	Default
id	integer	yes	PK	nextval('payment_allocations_id_seq'::re
payment_id	integer	yes	FK → payments	
invoice_id	integer	yes	FK → invoices	
amount	bigint	yes		
type	enum allocation_type	yes		'AUTO'::allocation_type
is_reversed	boolean	yes		false
reversed_at	timestamptz			
created_by	integer		FK → users	
created_at	timestamptz	yes		now()

Constraints:

- **CHECK** payment_allocations_amount_ck: CHECK ((amount > 0))

Indexes:

- `payment_allocations_invoice_idx`: btree (invoice_id)
- `payment_allocations_payment_idx`: btree (payment_id)

payment_proofs

Column	Type	Required	Key	Default
id	integer	yes	PK	<code>nextval('payment_proofs_id_seq'::regclas</code>
subscriber_id	integer	yes	FK → subscribers	
payment_id	integer		FK → payments	
method	enum payment_method	yes		<code>'GCASH'::payment_method</code>
reference_no	varchar(80)	yes		
sender_name	varchar(120)	yes		
sender_number	varchar(20)	yes		
amount	bigint	yes		
transaction_date	date	yes		
notes	text			
stored_name	varchar(80)	yes		
file_name	varchar(200)	yes		
mime_type	varchar(60)	yes		
file_size	integer	yes		
sha256	char(64)	yes		
status	enum proof_status	yes		<code>'PENDING'::proof_status</code>
submitted_by	integer		FK → users	
submitted_at	timestamptz	yes		<code>now()</code>
reviewed_by	integer		FK → users	
reviewed_at	timestamptz			
rejection_reason	text			

Constraints:

- **CHECK** `payment_proofs_ck`: CHECK (((amount > 0) AND ((status <> 'VERIFIED'::proof_status) OR ((payment_id IS NOT NULL) AND (reviewed_by IS NOT NULL) AND (reviewed_at IS NOT NULL)))))

Indexes:

- `payment_proofs_reference_idx`: btree (upper((reference_no)::text))
- `payment_proofs_status_idx`: btree (status, submitted_at)
- `payment_proofs_subscriber_idx`: btree (subscriber_id)

payment_reversals

Column	Type	Required	Key	Default
id	integer	yes	PK	<code>nextval('payment_reversals_id_seq'::regc</code>
reversal_no	varchar(20)	yes		
payment_id	integer	yes	FK → payments	
amount	bigint	yes		
reason	text	yes		
reversed_by	integer	yes	FK → users	
reversed_at	timestamptz	yes		<code>now()</code>

Constraints:

- **UNIQUE** `payment_reversals_paymentId_unique`: UNIQUE (payment_id)
- **UNIQUE** `payment_reversals_reversalNo_unique`: UNIQUE (reversal_no)

payments

Column	Type	Required	Key	Default
id	integer	yes	PK	<code>nextval('payments_id_seq'::regclass)</code>
subscriber_id	integer	yes	FK → subscribers	
payment_date	date	yes		
paid_at	timestamptz	yes		<code>now()</code>
amount	bigint	yes		
method	enum payment_method	yes		

Column	Type	Required	Key	Default
reference_no	varchar(80)			
notes	text			
status	enum payment_status	yes		'POSTED'::payment_status
unapplied_amount	bigint	yes		0
received_by	integer		FK → users	
collector_id	integer		FK → collectors	
collection_batch_id	integer		FK → collection_batches	
idempotency_key	uuid			
created_at	timestamptz	yes		now()

Constraints:

- **CHECK** payments_amount_ck: CHECK (((amount > 0) AND (unapplied_amount >= 0) AND (unapplied_amount <= amount)))
- **UNIQUE** payments_idempotencyKey_unique: UNIQUE (idempotency_key)

Indexes:

- payments_batch_idx: btree (collection_batch_id)
- payments_collector_idx: btree (collector_id)
- payments_date_idx: btree (payment_date)
- payments_gcash_reference_uq (unique): btree (upper((reference_no)::text)) WHERE ((method = 'GCASH'::payment_method) AND (status = 'POSTED'::payment_status))
- payments_reference_idx: btree (reference_no)
- payments_subscriber_idx: btree (subscriber_id, payment_date)

permissions

Column	Type	Required	Key	Default
id	integer	yes	PK	nextval('permissions_id_seq'::regclass)
code	varchar(60)	yes		
description	text	yes		'::text

Constraints:

- **UNIQUE** permissions_code_unique: UNIQUE (code)

receipts

Column	Type	Required	Key	Default
id	integer	yes	PK	nextval('receipts_id_seq'::regclass)
receipt_no	varchar(20)	yes		
payment_id	integer	yes	FK → payments	
status	enum receipt_status	yes		'ISSUED'::receipt_status
issued_by	integer		FK → users	
issued_at	timestamptz	yes		now()
voided_by	integer		FK → users	
voided_at	timestamptz			
void_reason	text			
print_count	integer	yes		0

Constraints:

- **UNIQUE** receipts_paymentId_unique: UNIQUE (payment_id)
- **UNIQUE** receipts_receiptNo_unique: UNIQUE (receipt_no)

Indexes:

- receipts_status_idx: btree (status)

reconnection_records

Column	Type	Required	Key	Default
id	integer	yes	PK	nextval('reconnection_records_id_seq'::r
service_account_id	integer	yes	FK → service_accounts	
suspension_id	integer		FK → suspension_records	
request_date	date	yes		
completion_date	date			
fee_amount	bigint	yes		0

Column	Type	Required	Key	Default
fee_invoice_id	integer		FK → invoices	
technician_user_id	integer		FK → users	
status	enum reconnection_status	yes		'REQUESTED'::reconnection_status
requested_by	integer		FK → users	
completed_by	integer		FK → users	
notes	text			
created_at	timestamptz	yes		now()

Indexes:

- `reconnection_records_account_idx: btree (service_account_id)`
- `reconnection_records_status_idx: btree (status)`

role_permissions

Column	Type	Required	Key	Default
role_id	integer	yes	PK, FK → roles	
permission_id	integer	yes	PK, FK → permissions	

Indexes:

- `role_permissions_role_id_permission_id_pk (unique): btree (role_id, permission_id)`

roles

Column	Type	Required	Key	Default
id	integer	yes	PK	nextval('roles_id_seq'::regclass)
code	varchar(40)	yes		
name	varchar(80)	yes		
description	text	yes		'':text

Constraints:

- **UNIQUE** `roles_code_unique: UNIQUE (code)`

service_accounts

Column	Type	Required	Key	Default
id	integer	yes	PK	nextval('service_accounts_id_seq'::regcl
account_no	varchar(20)	yes		
subscriber_id	integer	yes	FK → subscribers	
plan_id	integer	yes	FK → service_plans	
address_id	integer	yes	FK → subscriber_addresses	
activation_date	date	yes		
billing_start_date	date	yes		
due_day	integer	yes		
current_rate	bigint	yes		
monthly_discount	bigint	yes		0
status	enum service_account_status	yes		'ACTIVE'::service_account_status
collector_id	integer		FK → collectors	
notes	text			
created_by	integer		FK → users	
created_at	timestamptz	yes		now()
updated_at	timestamptz	yes		now()

Constraints:

- **CHECK** `service_accounts_ck: CHECK (((due_day >= 1) AND (due_day <= 28)) AND (current_rate > 0) AND (monthly_discount >= 0) AND (monthly_discount <= current_rate)))`
- **UNIQUE** `service_accounts_accountNo_unique: UNIQUE (account_no)`

Indexes:

- `service_accounts_collector_idx: btree (collector_id)`
- `service_accounts_plan_idx: btree (plan_id)`
- `service_accounts_status_idx: btree (status)`
- `service_accounts_subscriber_idx: btree (subscriber_id)`

service_events

Column	Type	Required	Key	Default
id	integer	yes	PK	nextval('service_events_id_seq'::regclas
service_account_id	integer	yes	FK → service_accounts	
event_type	varchar(40)	yes		
from_status	varchar(20)			
to_status	varchar(20)			
description	text	yes		
effective_date	date	yes		
created_by	integer		FK → users	
created_at	timestamptz	yes		now()

Indexes:

- service_events_account_idx: btree (service_account_id, effective_date)

service_plans

Column	Type	Required	Key	Default
id	integer	yes	PK	nextval('service_plans_id_seq'::regclas
code	varchar(20)	yes		
name	varchar(100)	yes		
service_type_id	integer	yes	FK → service_types	
monthly_price	bigint	yes		
installation_fee	bigint	yes		0
reconnection_fee	bigint	yes		0
speed_mbps	integer			
channel_count	integer			
description	text			
is_active	boolean	yes		true
created_at	timestamptz	yes		now()
updated_at	timestamptz	yes		now()

Constraints:

- CHECK service_plans_price_ck: CHECK (((monthly_price > 0) AND (installation_fee >= 0) AND (reconnection_fee >= 0)))
- UNIQUE service_plans_code_unique: UNIQUE (code)

service_types

Column	Type	Required	Key	Default
id	integer	yes	PK	nextval('service_types_id_seq'::regclass
code	enum service_type_code	yes		
name	varchar(60)	yes		

Constraints:

- UNIQUE service_types_code_unique: UNIQUE (code)

sessions

Column	Type	Required	Key	Default
id	uuid	yes	PK	gen_random_uuid()
user_id	integer	yes	FK → users	
token_hash	char(64)	yes		
client_name	varchar(80)			
ip	varchar(64)			
created_at	timestamptz	yes		now()
last_seen_at	timestamptz	yes		now()
expires_at	timestamptz	yes		
locked_at	timestamptz			
revoked_at	timestamptz			

Constraints:

- UNIQUE sessions_tokenHash_unique: UNIQUE (token_hash)

Indexes:

- `sessions_user_idx: btree (user_id)`

subscriber_addresses

Column	Type	Required	Key	Default
id	integer	yes	PK	nextval('subscriber_addresses_id_seq'::r
subscriber_id	integer	yes	FK → subscribers	
label	varchar(40)			
line1	varchar(160)	yes		
barangay	varchar(80)	yes		
city	varchar(80)	yes		
province	varchar(80)	yes		
landmark	varchar(160)			
is_primary	boolean	yes		false
created_at	timestamptz	yes		now()

Indexes:

- `subscriber_addresses_primary_uq (unique): btree (subscriber_id) WHERE is_primary`
- `subscriber_addresses_subscriber_idx: btree (subscriber_id)`
- `subscriber_addresses_trgm_idx: gin (((((((line1)::text || ' '::text) || (barangay)::text) || ' '::text) || (city)::text)) gin_trgm_ops)`

subscribers

Column	Type	Required	Key	Default
id	integer	yes	PK	nextval('subscribers_id_seq'::regclass)
account_no	varchar(20)	yes		
first_name	varchar(80)	yes		
last_name	varchar(80)	yes		
full_name	varchar(170)	yes		
phone	varchar(20)	yes		
alt_phone	varchar(20)			
email	varchar(160)			
due_day	integer	yes		15
status	enum subscriber_status	yes		'ACTIVE'::subscriber_status
collection_area_id	integer		FK → collection_areas	
collector_id	integer		FK → collectors	
route_sequence	integer			
notes	text			
created_by	integer		FK → users	
created_at	timestamptz	yes		now()
updated_at	timestamptz	yes		now()

Constraints:

- CHECK** `subscribers_due_day_ck: CHECK (((due_day >= 1) AND (due_day <= 28)))`
- UNIQUE** `subscribers_accountNo_unique: UNIQUE (account_no)`

Indexes:

- `subscribers_area_idx: btree (collection_area_id)`
- `subscribers_collector_idx: btree (collector_id)`
- `subscribers_name_idx: btree (full_name)`
- `subscribers_name_trgm_idx: gin (full_name gin_trgm_ops)`
- `subscribers_phone_idx: btree (phone)`
- `subscribers_phone_trgm_idx: gin (phone gin_trgm_ops)`
- `subscribers_status_idx: btree (status)`

suspension_records

Column	Type	Required	Key	Default
id	integer	yes	PK	nextval('suspension_records_id_seq'::reg
service_account_id	integer	yes	FK → service_accounts	
reason	text	yes		
effective_date	date	yes		

Column	Type	Required	Key	Default
arrears_at_suspension	bigint	yes		0
approved_by	integer		FK → users	
notes	text			
is_active	boolean	yes		true
lifted_at	timestamptz			
created_at	timestamptz	yes		now()

Indexes:

- `suspension_records_account_idx: btree (service_account_id)`

user_roles

Column	Type	Required	Key	Default
user_id	integer	yes	PK, FK → users	
role_id	integer	yes	PK, FK → roles	

Indexes:

- `user_roles_user_id_role_id_pk (unique): btree (user_id, role_id)`

users

Column	Type	Required	Key	Default
id	integer	yes	PK	nextval('users_id_seq'::regclass)
username	varchar(40)	yes		
full_name	varchar(120)	yes		
email	varchar(160)			
password_hash	text	yes		
is_active	boolean	yes		true
failed_login_count	integer	yes		0
locked_until	timestamptz			
last_login_at	timestamptz			
created_at	timestamptz	yes		now()
updated_at	timestamptz	yes		now()

Constraints:

- **UNIQUE** `users_username_unique: UNIQUE (username)`